

Tổng hợp kiến thức Bash Script và Cron

Tài liệu hệ thống hóa nội dung ghi chú: cấu trúc file Bash, biến, đối số dòng lệnh, array, điều kiện, vòng lặp, function và cronjob.

Mục tiêu tài liệu

Giúp ôn nhanh cú pháp Bash cơ bản và cách lên lịch chạy script bằng cron. Nội dung được chia theo danh mục để dễ học, dễ tra cứu và dễ mở rộng khi làm bài thực hành.

Mục lục

- 1. Tổng quan về Bash Script
- 2. Cấu trúc file Bash
- 3. Đối số dòng lệnh
- 4. Biến và cách gán giá trị
- 5. Dấu nháy và command substitution
- 6. Array trong Bash
- 7. Cấu trúc điều kiện if
- 8. Vòng lặp trong Bash
- 9. Các thao tác thường dùng với array
- 10. Tính toán trong Bash
- 11. Function trong Bash
- 12. Cron và crontab
- 13. Cấu trúc cronjob
- 14. Tóm tắt cú pháp nhanh

1. Tổng quan về Bash Script

Bash script là file chứa nhiều câu lệnh shell được viết theo thứ tự. Khi chạy script, Bash sẽ đọc và thực thi từng dòng lệnh giống như khi bạn gõ từng command trong terminal.

Bash thường được dùng để tự động hóa các công việc như xử lý file, chạy chương trình, tạo thư mục, lọc dữ liệu, backup, hoặc lên lịch tác vụ bằng cron.

2. Cấu trúc file Bash

Một file Bash cơ bản thường gồm hai phần chính:

- Shebang: dòng đầu tiên cho hệ điều hành biết script sẽ được chạy bằng chương trình nào.
- Nội dung script: mỗi dòng thường là một command hoặc một cấu trúc điều khiển.

```
#!/usr/bin/bash
```

```
echo "Hello Bash"
mkdir -p output
ls -la
```

Lưu ý về shebang

Trong ghi chú ban đầu có ghi `#!/usr/bin/bash`. Cách phổ biến và an toàn hơn trên Linux là `#!/usr/bin/bash` hoặc `#!/bin/bash`, tùy vị trí Bash trên máy.

3. Đối số dòng lệnh

Đối số dòng lệnh là các giá trị truyền vào khi chạy script. Bash cung cấp sẵn các biến đặc biệt để lấy những giá trị này.

Ký hiệu	Ý nghĩa
\$0	Tên script đang được chạy.
\$1, \$2, ...	Đối số thứ 1, thứ 2, ... truyền vào script.
\$#	Số lượng đối số được truyền vào.
\$@	Toàn bộ đối số, thường dùng khi muốn duyệt qua tất cả arguments.

```
#!/usr/bin/bash
```

```
echo "Tên script: $0"
echo "Đối số 1: $1"
echo "Đối số 2: $2"
echo "Số lượng đối số: $#"
```

```
echo "Tất cả đối số: $@"
```

4. Biến và cách gán giá trị

Biến trong Bash dùng để lưu giá trị. Khi gán biến, không được có khoảng trắng quanh dấu bằng.

```
var1="var"
echo "$var1"
```

Khi gọi giá trị của biến, cần dùng dấu \$ trước tên biến để Bash hiểu đây là biến chứ không phải text thường.

```
name="Thanh"
echo "Xin chào $name"
```

Lỗi thường gặp

Viết `var1 = "var"` là sai trong Bash vì có khoảng trắng quanh dấu =. Cách đúng là `var1="var"`.

5. Dấu nháy và command substitution

Bash xử lý dấu nháy khác nhau tùy loại dấu nháy được sử dụng.

Cú pháp	Tên gọi	Tác dụng
'text'	Nháy đơn	Giữ nguyên nội dung như raw text, không phân tích biến.
"text"	Nháy đôi	Cho phép phân tích biến và một số ký tự đặc biệt.
`command` hoặc \$(command)	Command substitution	Thực thi command rồi lấy output gán vào biến hoặc dùng trực tiếp.

```
name="Thanh"
echo 'Hello $name'  # In nguyên: Hello $name
echo "Hello $name"  # In: Hello Thanh
```

```
today=$(date)
echo "Hôm nay là: $today"
```

6. Array trong Bash

Bash hỗ trợ hai kiểu mảng cơ bản: mảng thường và mảng key-value.

6.1. Mảng thường

Mảng thường lưu các phần tử theo chỉ số số nguyên, bắt đầu từ 0. Các phần tử thường được phân tách bằng khoảng trắng.

```
declare -a my_array
numbers=(1 2 3 4 5)

echo "Phần tử đầu tiên: ${numbers[0]}"
```

```
# Gán lại phần tử không dùng dấu $ ở bên trái
numbers[0]=100
```

6.2. Mảng key-value

Mảng key-value còn gọi là associative array. Kiểu này dùng key thay vì chỉ số số nguyên.

```
declare -A user
user[name]="Thanh"
user[role]="Student"

echo "Tên: ${user[name]}"
```

7. Cấu trúc điều kiện if

Cấu trúc if dùng để chạy code theo điều kiện. Khi dùng [condition], bắt buộc có khoảng trắng sau [và trước].

```
if [ condition ]; then
    # code chạy khi condition đúng
else
    # code chạy khi condition sai
fi
```

7.1. So sánh số

```
if (( $2 > 10 )); then
    echo "Đối số thứ 2 lớn hơn 10"
fi
```

7.2. Kiểm tra file và thư mục

Flag	Ý nghĩa
-e file	File hoặc thư mục tồn tại.
-f file	Là file thường.
-d dir	Là thư mục.

```
if [ -f "data.csv" ]; then
    echo "data.csv là file thường"
fi
```

7.3. Kết hợp nhiều điều kiện

Có thể dùng [[... && ...]] hoặc [[... || ...]] để kết hợp nhiều điều kiện.

```
if [[ -f "data.csv" && -e "output" ]]; then
    echo "File data.csv tồn tại và output cũng tồn tại"
fi
```

8. Vòng lặp trong Bash

8.1. For duyệt list

```
for item in list
do
    echo "$item"
done
```

8.2. For theo range

```
for x in {1..5}
do
    echo "$x"
done
```

8.3. For kiểu C

```
for (( start; condition; update ))
do
    command
done
```

8.4. For duyệt file theo pattern

```
for file in books/*.txt
do
    echo "$file"
done
```

8.5. While

```
while [ condition ]
do
    command
done
```

9. Các thao tác thường dùng với array

Cú pháp	Ý nghĩa
<code>\${!array[@]}</code>	Lấy toàn bộ key hoặc index của mảng.
<code>\${array[@]}</code>	Lấy toàn bộ phần tử của mảng.
<code>\${array[0]}</code>	Lấy phần tử đầu tiên.
<code>\${#array[@]}</code>	Lấy số lượng phần tử.
<code>\${array[@]:start:length}</code>	Cắt mảng theo vị trí bắt đầu và số lượng phần tử.
<code>numbers+=(10)</code>	Thêm phần tử 10 vào cuối mảng.

```
numbers=(1 2 3 4 5)
echo "Tất cả phần tử: ${numbers[@]}"
echo "Số lượng phần tử: ${#numbers[@]}"
echo "Phần tử đầu tiên: ${numbers[0]}"
echo "Slice: ${numbers[@]:1:3}"
numbers+=(10)
```

10. Tính toán trong Bash

Bash có nhiều cách tính toán. Một số cách phổ biến gồm `expr`, `arithmetic expansion` và `bc`.

Cách	Dùng cho	Ví dụ
------	----------	-------

expr	Số nguyên	expr 5 + 5
\$((...))	Số nguyên	\$((5 + 5))
bc	Số thực	echo "scale=2; 10 / 3" bc

```
a=$((5 + 5))
echo "$a"
```

```
echo "scale=2; 10 / 3" | bc
```

Lưu ý

`$((5 + 5))` là arithmetic expansion của shell, dùng để tính toán số nguyên. Với số thực, nên dùng `bc`.

11. Function trong Bash

Function giúp gom một nhóm command thành một khối có thể gọi lại nhiều lần.

```
print_text() {
    echo "Hello from function"
}
```

```
print_text
```

Cũng có thể khai báo bằng từ khóa `function`:

```
function print_text {
    echo "Hello from function"
}
```

11.1. Trả dữ liệu từ function

Trong Bash, cách thường dùng để function “trả về dữ liệu” là `echo` kết quả, sau đó bên ngoài dùng `$()` để hứng output.

```
convert_temp() {
    local temp_f="$1"
    echo "scale=2; ($temp_f - 32) * 5 / 9" | bc
}
```

```
converted=$(convert_temp 30)
echo "$converted"
```

Về phạm vi biến

Biến trong Bash mặc định thường có phạm vi global trong script. Khi viết function, nên dùng `local` cho biến chỉ dùng bên trong function để tránh ghi đè nhầm biến bên ngoài.

12. Cron và crontab

cron là chương trình chạy nền trên hệ thống Unix/Linux/macOS. Nó dùng để tự động chạy command hoặc script theo lịch định sẵn.

cron đọc lịch từ một file gọi là crontab. Trong crontab, mỗi dòng là một cronjob. Mỗi cronjob mô tả thời điểm chạy và command cần chạy.

```
***** command
```

Cron thường được dùng cho các tác vụ lặp lại như backup dữ liệu, chạy script xử lý dữ liệu hằng ngày, dọn log, gửi report, hoặc đồng bộ hệ thống.

13. Cấu trúc cronjob

Một cronjob cơ bản có dạng:

```
***** /path/to/script.sh
```

Năm dấu * đầu tiên đại diện cho năm cột thời gian.

Vị trí	Ý nghĩa	Giá trị
1	Phút	0-59
2	Giờ	0-23
3	Ngày trong tháng	1-31
4	Tháng	1-12
5	Ngày trong tuần	0-7

13.1. Ví dụ cronjob

```
# Chạy script mỗi ngày lúc 2 giờ sáng
0 2 * * * /home/user/backup.sh
```

```
# Chạy mỗi 5 phút
*/5 * * * * /home/user/check_status.sh
```

```
# Chạy vào 8 giờ sáng thứ Hai hằng tuần
0 8 * * 1 /home/user/weekly_report.sh
```

14. Tóm tắt cú pháp nhanh

Chủ đề	Cú pháp / Ghi nhớ
Shebang	<code>#!/usr/bin/bash</code>
Gán biến	<code>name="Thanh";</code> không có khoảng trắng quanh dấu <code>=</code>
Gọi biến	<code>echo "\$name"</code>
Đổi số dòng lệnh	<code>\$0, \$1, \$2, \$#, \$@</code>
Mảng thường	<code>numbers=(1 2 3 4 5)</code>
Mảng key-value	<code>declare -A user</code>
If	<code>if [condition]; then ... else ... fi</code>
So sánh số	<code>((a > b))</code>
Kiểm tra file	<code>-e, -f, -d</code>
For	<code>for item in list; do ... done</code>
While	<code>while [condition]; do ... done</code>

Function	name() { ... }
Trả dữ liệu function	echo kết quả, húng bằng \$(function_name)
Cronjob	* * * * * command